

Anexo_Codigo_Limpio

Título: Informe de Refactorización

1. Introducción

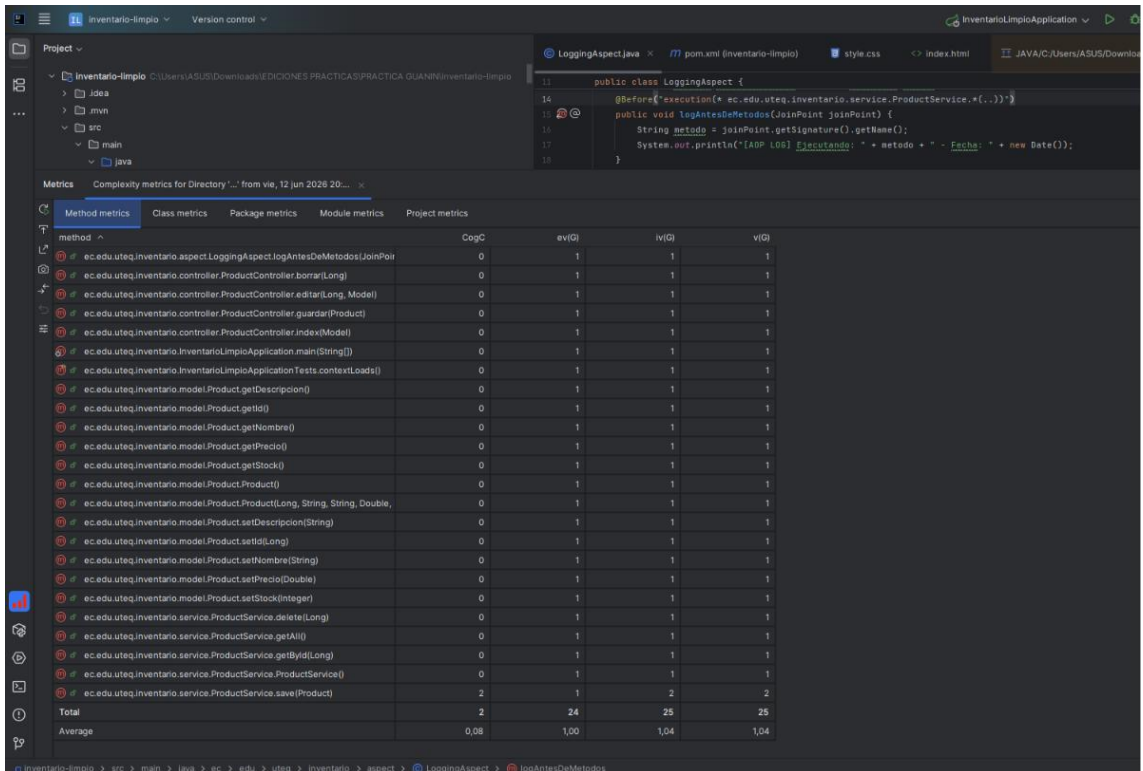
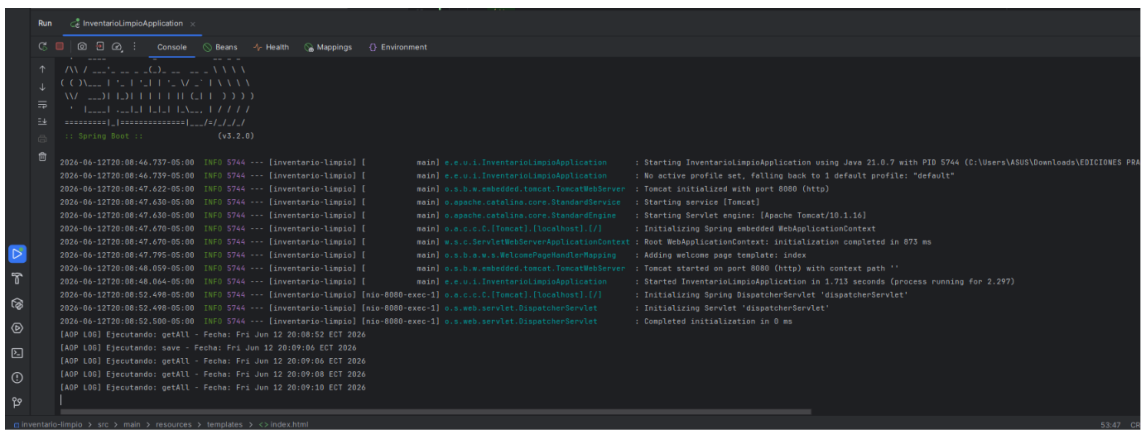
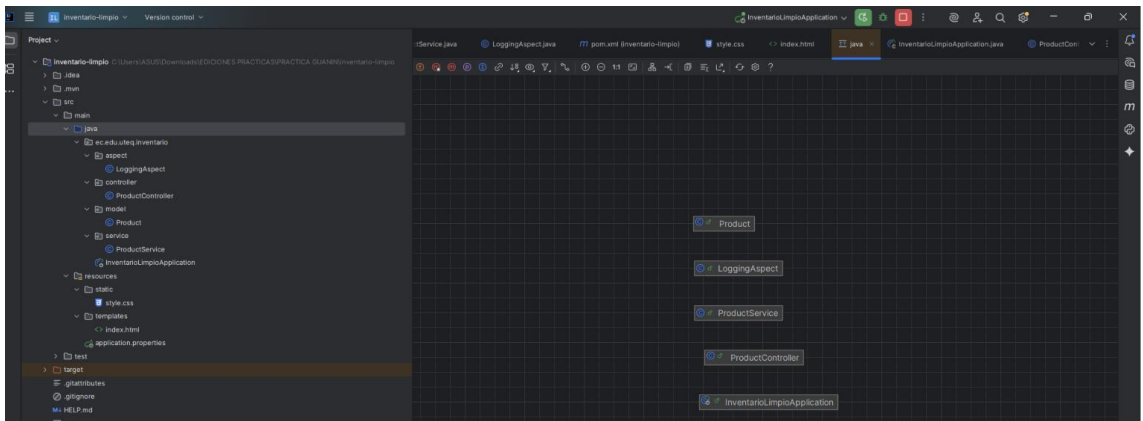
Este documento describe la transformación del sistema mediante un proceso de refactorización. Se han aplicado patrones de diseño y técnicas avanzadas como la Programación Orientada a Aspectos (AOP) para lograr un código limpio, modular y profesional.

2. Mejoras Arquitectónicas Implementadas

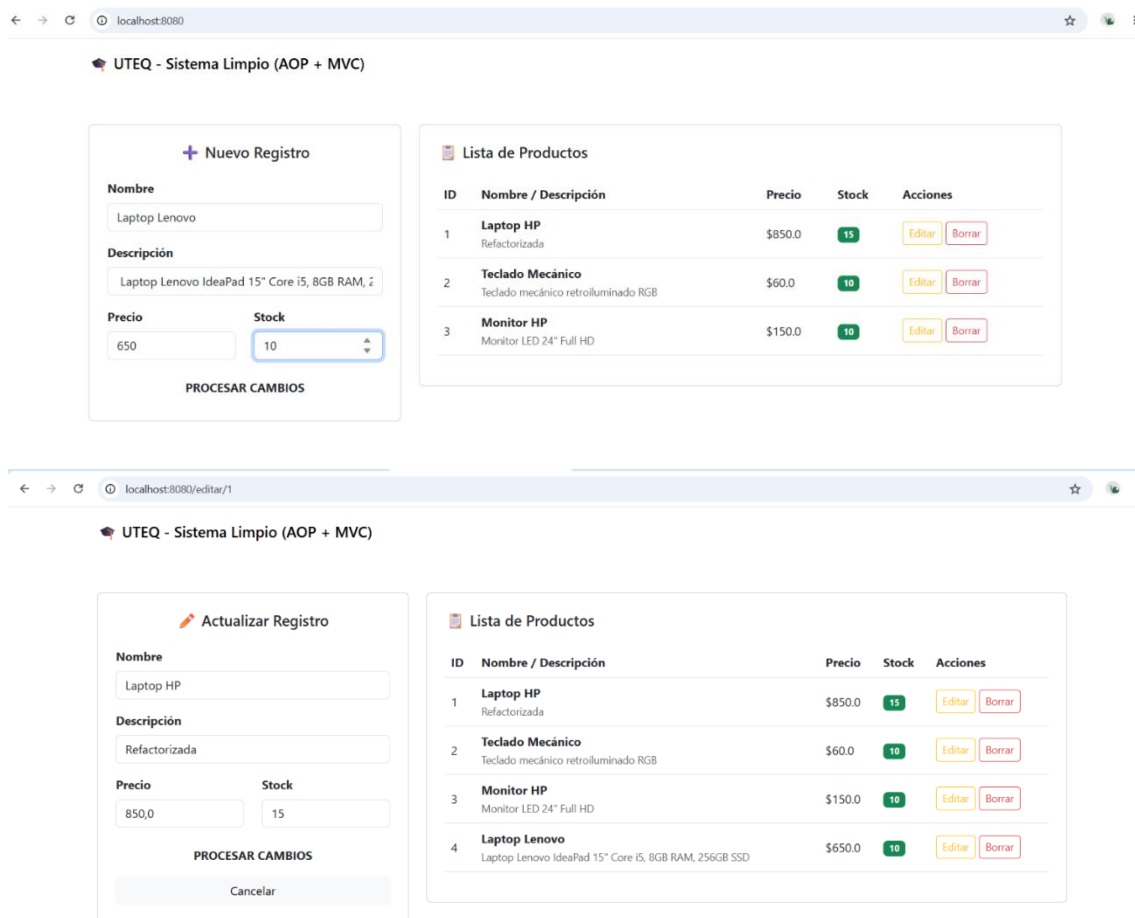
- 2.1. Adopción del Patrón MVC:**
Se dividió el proyecto en paquetes especializados para separar las responsabilidades:
 - Model:** Clase Product independiente.
 - Service:** Donde reside la lógica de negocio y gestión de datos.
 - Controller:** Encargado únicamente de gestionar las peticiones HTTP y retornar las vistas.
- 2.2. Programación Orientada a Aspectos (AOP):**
Se implementó la clase LoggingAspect. Esta técnica permite interceptar de forma automática los métodos de la capa de servicio para generar logs de auditoría, eliminando la necesidad de escribir código repetido en los controladores.
- 2.3. Inyección de Dependencias:**
Se utiliza la anotación @Autowired para comunicar las capas de forma desacoplada, facilitando futuras pruebas unitarias.

3. Cuadro Comparativo de Métricas (Antes vs. Después)

Métrica	Proyecto Original (Sucio)	Proyecto Refactorizado (Limpio)
Líneas de Código (Clase Principal)	~120 líneas (God Object)	~30 líneas (Controller limpio)
Complejidad Ciclomática	Alta (Lógica enredada)	Baja (Métodos pequeños y claros)
Acoplamiento	Alto (Controlador depende de todo)	Bajo (Uso de interfaces y @Autowired)
Mantenibilidad	Difícil y riesgosa	Fácil y escalable



Interfaz



Conclusión

La refactorización del Sistema de Inventario UTEQ ha permitido una transición exitosa desde un diseño monolítico hacia una arquitectura modular, escalable y profesional. Al adoptar el patrón MVC (Modelo-Vista-Controlador), se ha logrado un desacoplamiento efectivo donde la lógica de negocio, la gestión de datos y la interfaz de usuario operan de forma independiente. Esto garantiza que cualquier cambio futuro en la base de datos o en la interfaz no afecte la estabilidad del núcleo del sistema.

Uno de los avances más significativos es la implementación de la Programación Orientada a Aspectos (AOP) a través de la clase `LoggingAspect`. Esta técnica ha permitido centralizar las tareas secundarias (como el registro de logs), eliminando la redundancia de código en los controladores y mejorando la legibilidad del proyecto. Gracias a esto, el código fuente es ahora más limpio y se enfoca exclusivamente en sus responsabilidades principales.

Finalmente, el uso de inyección de dependencias y la reducción de la complejidad ciclomática aseguran que el software sea fácil de testear y mantener. Esta versión refactorizada no solo es visualmente más organizada, sino que es técnicamente robusta, cumpliendo con los estándares de la industria y minimizando significativamente la deuda técnica del proyecto.